TEC | Tecnológico de Costa Rica

IC8071 - Seguridad de software

Análisis de Riesgo de Arquitectura

Proyecto 3

Estudiantes:

Oscar Roni - 2023150387

Kendall Piedra - 2023164712

Docente:

Dr. Herson Esquivel Vargas

Año:

2025

Proyecto a analizar y su arquitectura..... 4
Prompt para generar la arquitectura..... 4
Selección del Stack Tecnológico..... 4
Diagrama de Arquitectura de Alto Nivel..... 5
Descripción textual de partes y conexiones..... 6
1. Capa de Cliente (Edge)..... 6
A. App Conductor (Flutter)..... 6
B. Panel Web (React.js)..... 7
2. Capa de Entrada (API Gateway / Load Balancer)..... 7
3. Capa de Aplicación (Backend NestJS)..... 7
4. Capa de Servicios de Ruteo (Dockerizado)..... 8
5. Capa de Datos..... 8
Estudio de la arquitectura..... 9
Componentes e interacciones principales..... 9
App Conductor (Flutter):..... 9
Panel Dispatcher:..... 9
Load Balancer:..... 10
Modulo Auth:..... 10
Módulo Ingesta:..... 11
Módulo Core:..... 11
Módulo reportes:..... 12
Instancia de OSRM:..... 12
Instancia VROOM:..... 12
Redis (Memoria Caché):..... 13
PostgreSQL (Persistencia):..... 13
Flujo de datos..... 13
1. Inicio de Jornada:..... 13
2. Asignación:..... 13
3. En Ruta:..... 14
4. Entrega:..... 14
Identificación de amenazas..... 14
Implementación del patrón en la arquitectura..... 15
App Conductor (Flutter):..... 15
Panel Dispatcher:..... 16
Load Balancer:..... 17
Modulo Auth:..... 18
Módulo Ingesta:..... 19
Módulo Core:..... 20
Módulo reportes:..... 21
Instancia de OSRM:..... 22
Instancia VROOM:..... 23
Redis (Memoria Caché):..... 24
PostgreSQL + PostGIS (Persistencia):..... 25
Tabla de resultados..... 26

Probabilidad de Compromiso..... 27
Análisis de Impacto..... 33
Contexto del análisis..... 33
Tabla de análisis de impacto de amenazas..... 33
Priorización de riesgos..... 36
Cálculo del riesgo..... 36
Tabla de riesgos..... 37
Plan de Mitigación de Riesgos..... 39
Mitigación par casos seleccionados en base al riesgo..... 39
Information Disclosure en PostgreSQL (37)..... 39
Info disclosure en módulo Auth (14)..... 39
Elevation of Privilege en VROOM (31)..... 40
Information Disclosure en Redis (33)..... 40
Tampering en PostgreSQL (35)..... 40
Elevation of privilege en módulo Auth (15)..... 41
Tampering en Core (18)..... 41
Info Disclosure en Panel Dispatcher (6)..... 41
information Disclosure en Reportes (23)..... 42
Elevation of Privilege en OSRM (28)..... 42
Repudiation en PostgreSQL (36)..... 42
Denial of Service en PostgreSQL (38)..... 43
Spoofing en módulo Auth (13)..... 43
Hallazgos..... 43

Proyecto a analizar y su arquitectura

Prompt para generar la arquitectura

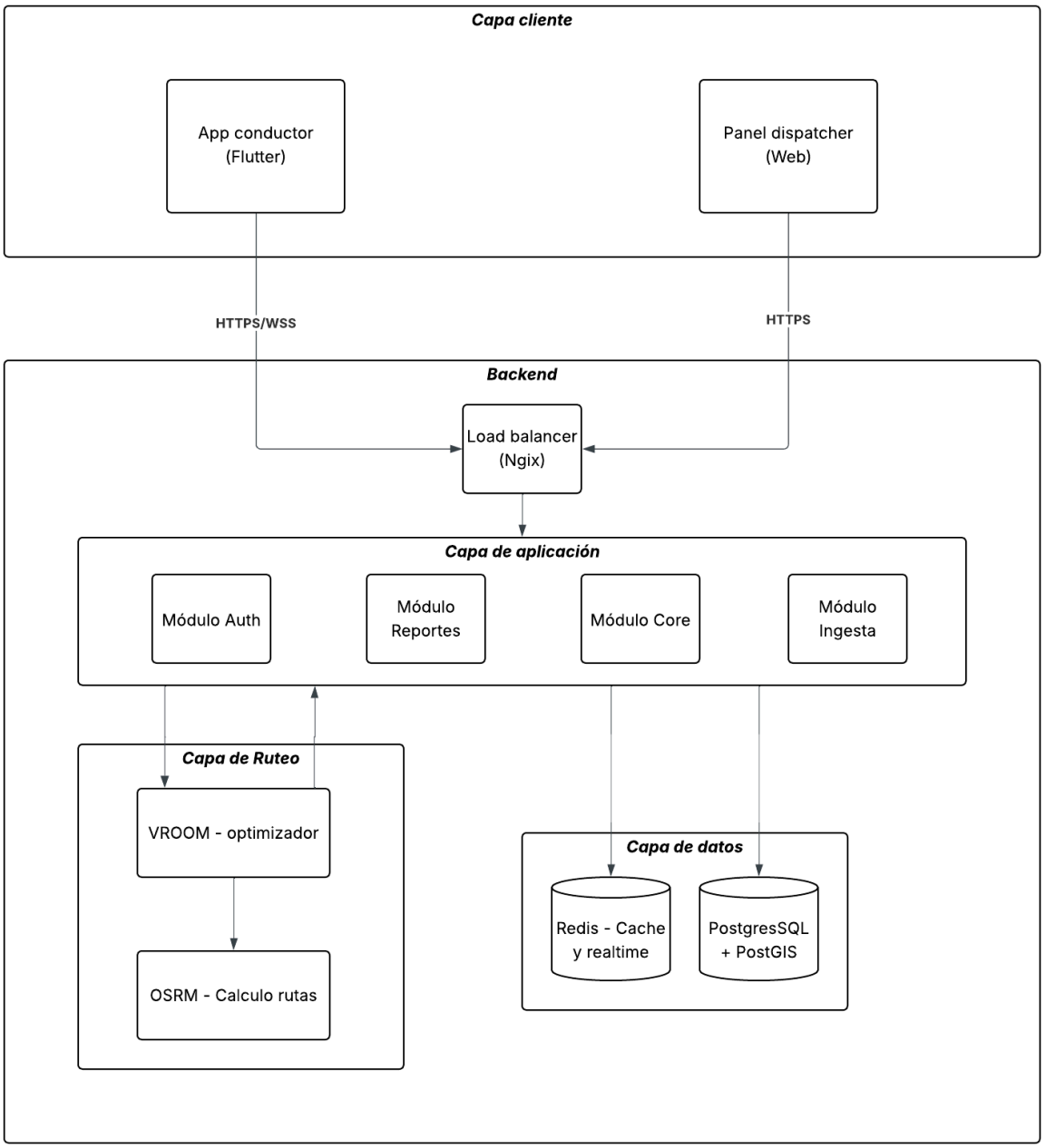
> Ponte en los zapatos de un ingeniero de software que trabaja en una plataforma muy simple de gestión de flotas de vehículos de reparto para pequeñas empresas que se va a realizar en Costa Rica en la industria de Logística. Está diseñada para móviles y tiene como requerimientos:
>
> - Registro y gestión de vehículos y conductores.
> - Crear y asignar órdenes de reparto, así como la administración de estas y la confirmación de sus entregas.
> - Motor básico de ruteo que pueda agrupar entregas, reordenarlas y optimizar las rutas.
> - Registros y reportes de exceso de velocidad, frenadas bruscas, tiempo de motor encendido, consumo estimado, entregas por día, km por vehículo, tiempos de inactividad y consumo.
> - Roles y permisos que permitan el control de las acciones (Administrador, dispatcher, conductor).
>
> A partir de este contexto plantea una arquitectura ficticia para el proyecto que describa sus componentes, interacciones, flujos de datos. Esta tiene que ser completa y describir toda la plataforma, pero manteniéndolo lo más simple posible (que sea funcional y básica). Hazlo con un diagrama de alto nivel y con una descripción textual detallada de las diferentes partes que componen la arquitectura y sus conexiones. Escoge tú mismo las tecnologías a usar y todos sus detalles basado en el contexto.

Selección del Stack Tecnológico

- **Backend: Node.js con NestJS**. Es robusto, estructurado y excelente para manejar I/O asíncrono (perfecto para recibir coordenadas GPS de muchos choferes a la vez).
- **Base de Datos: PostgreSQL con la extensión PostGIS**. Indispensable para logística. Nos permite hacer consultas espaciales (ej. "¿Qué vehículo está a 5km de este punto?") de forma nativa.
- **Frontend Web (Admin/Dispatcher): React.js** con una librería de componentes ligera (Mantine o Material UI).
- **Mobile App (Conductores): Flutter**. Un solo código para Android (predominante en CR) e iOS.

- **Motor de Mapas/Ruteo: OSRM (Open Source Routing Machine) + VROOM**.
    - *Por qué:* Google Maps API es muy cara para startups. Usaremos **OpenStreetMap (OSM)** con OSRM auto-alojado para calcular rutas y VROOM para optimizarlas (el problema del vendedor viajero).
    - *Frontend Map:* **Mapbox GL JS** (capa gratuita generosa) o Leaflet.
- **Infraestructura:** AWS o DigitalOcean (App Platform).

Diagrama de Arquitectura de Alto Nivel



1. **Capa de Cliente:**
    - **App Conductor (Flutter):** Recolecta GPS, acelerómetro (frenadas), y gestiona el estado de las órdenes.
    - **Panel Web (React):** Donde el Dispatcher ve el mapa en vivo y crea rutas.
2. **Capa de Entrada (API Gateway / Load Balancer):** Nginx o el LB del proveedor cloud.
3. **Capa de Aplicación (Backend NestJS):**
    - *Módulo Auth:* JWT y Roles (RBAC).
    - *Módulo Ingesta:* Recibe "pings" de ubicación cada 5-10 segundos.
    - *Módulo Core:* Gestión de vehículos, choferes y CRUD de órdenes.
    - *Módulo Reportes:* Procesa los datos crudos a estadísticas.
4. **Capa de Servicios de Ruteo (Dockerizado):**
    - Instancia de **OSRM** (Calcula distancias y tiempos).
    - Instancia de **VROOM** (Optimiza: "Tengo 10 paradas, dime el mejor orden").
5. **Capa de Datos:**
    - **Redis:** Para caché de ubicación en tiempo real (muy rápido) y colas de tareas.
    - **PostgreSQL (PostGIS):** Persistencia de datos e histórico.

Descripción textual de partes y conexiones

1. Capa de Cliente (Edge)

Esta es la cara visible de la plataforma. Es donde ocurre la interacción humana y la recolección física de datos.

A. App Conductor (Flutter)

Es el "sensor" del sistema. No es solo una pantalla pasiva; es un recolector de datos activo.

- **Detalle Técnico:**
    - **Background Service:** La app corre un servicio en segundo plano (incluso con el teléfono bloqueado) que despierta al GPS cada ***X*** segundos.
    - **Buffer Local (SQLite):** Si el conductor entra en una "zona muerta" (sin señal 4G), la app **no** descarta los datos. Los guarda en una base de datos interna (SQLite). Apenas recupera conexión, hace un "batch upload" (subida por lotes) al servidor.
    - **Acelerómetro:** Lee los valores de fuerza G (ejes X, Y, Z) del hardware del celular. Un algoritmo simple filtra el ruido (baches) y detecta picos sostenidos (frenazos o acelerones).
- **Conexiones:**
    - Se conecta vía **HTTPS (REST)** para descargar su lista de entregas.
    - Se conecta vía **WSS (WebSocket)** o HTTP POST frecuentes para enviar la telemetría (ubicación).

B. Panel Web (React.js)

El centro de comando para el Dispatcher.

- **Detalle Técnico:**
  - **Estado Global:** Maneja el estado de la flota. No recarga la página completa; usa AJAX/Fetch para traer datos nuevos.
  - **Renderizado de Mapas:** Usa librería de **Leaflet**. Esta librería no pide el mapa a nuestro servidor, sino que pide las "tiles" (los dibujos de las calles) directamente a un servidor de mapas (como OpenStreetMap), ahorrando carga a nuestro backend. Solo le pide a nuestro backend los "puntos" (dónde están los camiones).
- **Conexiones:**
  - Mantiene un **Socket abierto** con el Backend. Cuando un camión se mueve, el servidor le "empuja" (push) la nueva coordenada y el ícono en el mapa se mueve suavemente.

2. Capa de Entrada (API Gateway / Load Balancer)

El portero del edificio. Aquí usamos **Nginx** (o el servicio de Balanceo de Carga de AWS/DigitalOcean).

- **Función Detallada:**
  - **Terminación SSL:** Aquí se desencripta el tráfico HTTPS. El certificado de seguridad vive aquí.
  - **Seguridad Básica:** Protege contra ataques DDOS simples y limita la cantidad de peticiones (Rate Limiting) para que un celular dañado no inunde el servidor.
  - **Enrutamiento:** Decide a dónde va el tráfico. Si la url es /api/... lo manda al Backend. Si es estático, sirve los archivos del Frontend.
- **Conexiones:**
  - Recibe todo el tráfico de la Capa de Cliente y lo pasa "limpio" a la Capa de Aplicación (Backend).

3. Capa de Aplicación (Backend NestJS)

El cerebro monolítico. Al usar **NestJS**, organizamos el código en "Módulos" lógicos dentro de un mismo servidor.

- **Módulo Auth (Seguridad):**
  - Valida quién eres. Cuando el conductor se loguea, este módulo genera un **JWT (JSON Web Token)**. Cada vez que la App envía una ubicación, adjunta este token. Si el token es falso o expiró, el backend rechaza el dato.
- **Módulo Ingesta (La Tubería de Datos):**
  - Este es el punto más estresado del sistema. Recibe cientos de coordenadas por minuto.
  - **Estrategia:** No escribe directamente en la base de datos (Postgres) porque es lento. Escribe primero en **Redis** (memoria RAM, ultra rápido) para que el

Dispatcher lo vea en vivo, y luego en cola una tarea para guardar en Postgres de forma asíncrona.

- **Módulo Core (Negocio):**
  - Aquí vive la lógica de "CRUD" (Crear, Leer, Actualizar, Borrar).
  - Ejemplo: Cuando se crea una orden, este módulo valida que la dirección exista y asigna un estado inicial "Pendiente".
- **Módulo Reportes:**
  - Corre procesos nocturnos o bajo demanda.
  - Le pide a Postgres: *"Suma todos los kilómetros del vehículo X entre las 8am y 5pm"*.
- **Conexiones:**
  - Habla con **Redis** para datos calientes (ahora).
  - Habla con **Postgres** para datos fríos (historial).
  - Habla con la **Capa de Ruteo** vía HTTP interno para pedir cálculos geométricos.

4. Capa de Servicios de Ruteo (Dockerizado)

Aquí externalizamos la matemática pesada. Son contenedores Docker que corren motores de código abierto.

- **Instancia de OSRM (Open Source Routing Machine):**
  - **Qué hace:** Es el mapa convertido en grafo matemático. Sabe que de la "Calle 5" a la "Avenida 2" hay 500 metros y se puede ir a 40km/h.
  - **Input:** Coordenada A y Coordenada B.
  - **Output:** Distancia exacta, tiempo estimado y la geometría (la línea azul que se dibuja en el mapa).
- **Instancia de VROOM (Vehicle Routing Open-Source Optimization Machine):**
  - **Qué hace:** Resuelve el "Problema del Vendedor Viajero" (TSP).
  - **Input:** "Tengo 1 camión y 20 entregas en San José".
  - **Proceso:** VROOM le pregunta a OSRM las distancias entre *todos* los puntos (Matriz de Costos) y luego usa algoritmos heurísticos para encontrar el orden más eficiente.
  - **Output:** "Visita el punto 5 primero, luego el 12, luego el 3...".
- **Conexiones:**
  - Recibe peticiones JSON del **Backend NestJS**.
  - **VROOM** hace peticiones HTTP internas a **OSRM**.

5. Capa de Datos

La memoria del sistema.

- **Redis (Memoria - Caché):**
  - **Uso:** Almacena la "Última Ubicación Conocida" de cada chofer.
  - **Por qué:** Cuando el Dispatcher carga el mapa, el backend consulta Redis (0.001ms de respuesta) en lugar de buscar en millones de registros en disco. También maneja los canales de Pub/Sub para los WebSockets.
- **PostgreSQL + PostGIS (Disco - Persistencia):**

    - **Uso:** Base de datos relacional robusta.
    - **PostGIS:** Es la extensión que permite tipos de datos como GEOMETRY(Point, 4326).
    - **Funciones Especiales:** Permite consultas como ST_DWithin(vehiculo_location, bodega_location, 1000). Esto traduce a: *"Dime si el vehículo está a menos de 1000 metros de la bodega"*. Sin PostGIS, tendrías que hacer fórmulas de trigonometría complejas manualmente en el código.
- **Conexiones:**
    - Solo el **Backend (NestJS)** tiene permiso para leer/escribir aquí. Nadie más toca la base de datos directamente por seguridad.

Estudio de la arquitectura

Componentes e interacciones principales

App Conductor (Flutter):

- Recopila datos constantemente, guarda datos sin conexión y contiene un acelerómetro que realiza cálculos constantemente.
- Sus interacciones son:
    - Adquiere datos desde un GPS en el dispositivo móvil.
    - Extrae valores desde el hardware del celular para cálculos de aceleración.
    - Recibe vía HTTP/HTTPS una lista de entregas .
    - Enviar datos de ubicación a través un WebSocket o HTTP POST al Load Balancer.
- Límites de Confianza:
    - GPS del dispositivo
        - Nivel de confianza: Bajo.
        - Los datos del GPS vienen de un componente externo y pueden ser manipulados.
    - Acelerómetro
        - Nivel de confianza: Medio.
        - El hardware es confiable, pero se mantiene en control del usuario y puede ser alterado.
    - Recepción vía HTTP de lista de entregas y envío de datos de ubicación
        - Nivel de confianza: Medio.
        - Es posible la modificación o intercepción de órdenes.

Panel Dispatcher:

- Muestra a administradores la flota en tiempo real, asignar rutas y visualizar reportes.
- Sus interacciones son:
    - Recibe datos de ubicación desde el backend.

    - Solicita información al backend mediante peticiones HTTP.
    - Descarga tiles de mapas desde servidores externos.
- Límites de confianza:
    - Navegador de usuario.
        - Nivel de confianza: Bajo.
        - El navegador puede estar comprometido y las peticiones pueden ser manipuladas.
    - Servidores externos.
        - Nivel de confianza: Bajo.
        - Fuente externa no controlada por la empresa.

Load Balancer:

- Terminación SSL, funciona como limitación del rate y enrutamiento hasta el backend.
- Sus interacciones son:
    - Recibe todo el tráfico desde la App Conductor y el Panel Dispatcher.
    - Redirige solicitudes hacia el backend.
- Límites de confianza:
    - Cliente externo.
        - Nivel de confianza: bajo.
        - Todos los datos externos pueden ser maliciosos.
    - Backend.
        - Nivel de confianza: medio.
        - El tráfico no está completamente inspeccionado, solo filtrado.

Modulo Auth:

- Autenticación y autorización de los usuarios del sistema por medio de tokens JWT y verificación de roles.
- sus interacciones son:
    - Recibe solicitudes de inicio de sesión desde el Panel dispatcher y el App que pasan por el Load balancer.
    - Emite tokens JWT a los usuarios autenticados.
    - Valida tokens en las peticiones entrantes al sistema con credenciales en la base de datos.
- Límites de confianza.
    - Recepción de credenciales.
        - Nivel de confianza: Bajo.
        - Los datos provienen directamente del usuario.
    - Validación de tokens.
        - Nivel de confianza: Medio.
        - El token es generado por el sistema, pero pasa por el usuario.
    - Consulta de credenciales en base de datos.
        - Nivel de confianza: Alto.
        - Es información provista por la base de datos interna.

Módulo Ingesta:

- Recibe y procesa datos de los conductores de manera rápida para telemetría en tiempo real y posterior almacenamiento en Postgres.
- Sus interacciones son:
    - Recibe peticiones frecuentes de la **App conductor (**a través de **load balancer).**
    - Valida el *JWT* con el **módulo auth.**
    - Escribe la última ubicación conocida de un conductor en **Redis**.
    - Crea una cola para escribir el historial en **Postgres.**
- **Límites de confianza:**
    - App conductor
        - Nivel de confianza: Bajo
        - Datos generados por un dispositivo fuera del control del sistema. Puede ser abusada o manipulada.
    - Redis
        - Nivel de confianza: Medio
        - Módulo interno pero contiene datos que no fueron validados rigurosamente.
    - Postgres
        - Nivel de confianza: Alto
        - Es un componente interno y se espera que la información haya sido validada con anterioridad.

Módulo Core:

- Módulo con lógica de negocio para crear, leer, actualizar y borrar información relacionada a las rutas, conductores, camiones y entregas.
- Sus interacciones son:
    - Recibe peticiones CRUD del **panel dispatcher.**
    - Lee, escribe y actualiza la base de datos **Postgres.**
    - Envía peticiones al **VROOM** para el cálculo de rutas e información relacionada.
    - Mantiene una conexión abierta para que el **panel dispatcher** pueda leer la posición más reciente de los conductores en **Redis**.
- Límites de confianza:
    - Panel dispatcher
        - Nivel de confianza: Bajo
        - Las peticiones pueden verse comprometidas y se necesitan validaciones.
    - Postgres
        - Nivel de confianza: Alto
        - Se espera que la info haya sido validada.
    - Redis
        - Nivel de confianza: media
        - La información almacenada viene directamente de la app móvil sin mucha validación. La info que lee puede ser comprometida o ser víctima de casos de abuso.

    - VROOM
        - Nivel de confianza: Alto
        - Componente interno con conexión solo a la propia capa de aplicación.

Módulo reportes:

- Realiza y guarda consultas en forma de reportes varios a la base de datos cuando haya baja demanda.
- Sus interacciones son:
    - Realiza consultas a **Postgres** para generar reportes
    - Retorna reportes según petición del **panel dispatcher.**
- Límites de confianza:
    - Postgres
        - Nivel de confianza: Alto
        - Se espera que la información almacenada haya sido validada anteriormente.
    - Panel dispatcher
        - Nivel de confianza: Medio
        - Existen validaciones previas de identidad por el módulo de autenticación para llegar hasta este módulo. Pero todavía puede recibir entradas problemáticas que deben ser validadas.

Instancia de OSRM:

- Instancia contenerizada de un mapa en forma de grafo para el cálculo de rutas.
- Sus interacciones son:
    - Recibe peticiones del **VROOM** de distancias, tiempos estimados y geometría.
- Límites de confianza
    - VROOM
        - Nivel de confianza: Alta
        - La VROOM depende de la OSRM, por lo que se le da gran acceso a su información

Instancia VROOM:

- Servicio de ruteo y optimización de entregas con base a info del OSRM.
- Sus interacciones son:
    - Recibe peticiones de rutas y cálculos del **módulo core**.
    - Solicita información y cálculos relacionados a las rutas al **OSRM.**
- Límites de confianza:
    - **Módulo core**
        - Nivel de confianza: Alto
        - Se prevé que la información de esta fuente interna sea validada.
    - **OSRM**
        - Nivel de confianza: Alto
        - Al ser un componente aislado de otros y con permisos de escritura limitados se confía en su información.

Redis (Memoria Caché):

- Memoria temporal que guarda los datos más recientes para fácil acceso.
- Sus interacciones son:
    - Recibe datos recientes de conductores a través de la capa de aplicación (**Módulo de ingesta**).
    - Se lee la información cuando un *Dispatcher* carga el mapa en la web a través de la **capa de aplicación (módulo Core)**.
- Límites de confianza:
    - Módulo de ingesta y módulo core
        - Nivel de confianza: Alto
        - Se espera que toda la información directa de la capa de aplicación sea validada ahí mismo.

PostgreSQL (Persistencia):

- Base de datos principal con extensión para datos y cálculos posicionales que almacena la información del sistema.
- Sus interacciones son:
    - Recibe un batch de datos de los conductores a través del **módulo ingesta** para guardarlos**.**
    - Recibe peticiones de lectura, escritura y modificación de datos del **módulo Core**.
    - Recibe consultas con cálculos del **módulo reportes**.
    - Recibe consultas de credenciales del **módulo Auth**.
- Límites de confianza:
    - Módulo de ingesta, módulo reportes, módulo Auth y módulo core
        - Nivel de confianza: Alto
        - Se espera que toda la información directa de la capa de aplicación sea validada ahí mismo.

Flujo de datos

1. Inicio de Jornada:

- El conductor abre la app y da tap en "Iniciar Turno".
- La información se pasa al backend por medio del load balancer.
- Se valida la sesión con el módulo Auth.
- El módulo core registra la hora (para reporte de inactividad).
- El módulo de ingesta empieza a escuchar coordenadas.

2. Asignación:

- El Dispatcher solicita al sistema agrupar entregas, calcular ruta y la asignación a un conductor.
- La solicitud HTTPS ingresa por el load balancer.
- Se pasa a la capa de aplicación donde se ejecuta en el módulo correspondiente.
- El componente de ruteo (VROOM) o la base de datos (Redis o Postgres) retorna lo solicitado.

- La respuesta regresa cada paso hasta llegar al dispatcher.

3. En Ruta:

- El conductor conduce.
- Su teléfono envía su ubicación cada X segundos al backend e información extra de otros eventos cuando ocurren.
- El módulo de ingesta guarda la info reciente en Redis.
- Se agrega a una cola.
- Cuando hay suficiente información en cola se guarda en Postgres (histórico).
- La información en vivo se envía de regreso al panel dispatcher por medio de web sockets.

4. Entrega:

- El conductor llega al cliente.
- Se solicita confirmación de la entrega.
- Se envía por HTTPS hacia el load balancer
- Se pasa a la capa de aplicación donde se evalúa en el módulo core.
- La base de datos (Redis o Postgres) guarda lo solicitado.

Identificación de amenazas

Se identifican las amenazas por medio del modelo estructurado STRIDE. Que valora los tipos de amenaza específicos: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

Letra	Amenaza	Viola
S	Spoofing (Suplantación)	Autenticación
T	Tampering (Manipulación)	Integridad
R	Repudiation (Repudio)	No repudio / auditoría
I	Information Disclosure	Confidencialidad
D	Denial of Service	Disponibilidad
E	Elevation of Privilege	Autorización

Implementación del patrón en la arquitectura

A cada amenaza encontrada con el patrón se le asigna un índice señalado por el número entre paréntesis. Este índice se va a utilizar en etapas posteriores del análisis para el estudio de su probabilidad, impacto, etc.

App Conductor (Flutter):

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(1)** El usuario puede modificar la ubicación antes de ser enviada mediante modificaciones del dispositivo.
T – Tampering	**(2)** Los datos recibidos por el acelerómetro pueden ser alterados desde el hardware para enviar falsa información.
R – Repudiation	**N/A**
I – Information Disclosure	**(3)** La lista de entregas puede ser interceptada si la comunicación no está debidamente protegida.
D – Denial of Service	**N/A**
E – Elevation of Privilege	**N/A**

Panel Dispatcher:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(4)** Es posible robar o reutilizar un JWT válido del panel para suplantar al usuario.
T – Tampering	**(5)** Las peticiones pueden ser modificadas antes del envío al backend.
R – Repudiation	**N/A**
I – Information Disclosure	**(6)** Si el canal http es interceptado la información puede filtrarse. **(7) L**os tiles de mapas consultados a servidores externos podrían revelar patrones de uso.
D – Denial of Service	**(8)** Se puede generar un cuello de botella por medio de una cantidad excesiva de peticiones simultáneas.
E – Elevation of Privilege	**N/A**

Load Balancer:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(9)** Al ser solicitudes externas y sin validar se puede recibir tráfico falsificado desde cualquier origen.
T – Tampering	**(10)** Es posible modificar campos o parámetros de consulta antes de la llegada al Load Balancer
R – Repudiation	**N/A**
I – Information Disclosure	**(11)** Si hay errores de configuración en el LB y no se protegen las salidas se puede filtrar rutas internas, puertos, mensajes de error, entre otras cosas por medio de mensajes de error.
D – Denial of Service	**(12)** Se está expuesto a intentos de DDoS como punto más cargado de flujo.
E – Elevation of Privilege	**N/A**

Modulo Auth:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(13)** Si un atacante toma un JWT válido, Auth le permitiría su acceso igualmente.
T – Tampering	**N/A**
R – Repudiation	**N/A**
I – Information Disclosure	**(14)** Si los mensajes de error del Auth son detallados pueden revelar información sobre las cuentas.
D – Denial of Service	**N/A**
E – Elevation of Privilege	**(15)** Es posible modificar el JWT para elevar el rol de un usuario.

Módulo Ingesta:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**N/A**
T – Tampering	**(16)** Es posible enviar datos falsos al módulo. Se pueden almacenar información manipulada que afecte a informes.
R – Repudiation	**N/A**
I – Information Disclosure	**N/A**
D – Denial of Service	**(17)** El proceso de validación puede generar retrasos si se le envía un datos demasiado grandes, muy anidados o inesperados por parte de la app.
E – Elevation of Privilege	**N/A**

Módulo Core:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**N/A**
T – Tampering	**(18)** Se puede recibir solicitudes con input malicioso o problemático tipo inyección SQL, XSS, caracteres especiales, comandos, etc...
R – Repudiation	**(19)** Uso de cuentas compartidas (tipo admin) o falta registro (logs) de peticiones al módulo.
I – Information Disclosure	**(20)** Se pueden devolver mensajes de error directos de las bases de datos y otros componentes.
D – Denial of Service	**N/A**
E – Elevation of Privilege	**N/A**

Módulo reportes:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**N/A**
T – Tampering	**(21)** Se puede alterar la petición HTTP para generar reportes oficiales con datos modificados dependiendo del tipo de reporte.
R – Repudiation	**(22)** Si no se registra quien hizo cual reporte se puede realizar consultas que degraden el rendimiento y negar haberlo hecho.
I – Information Disclosure	**(23)** Descarga y distribución de reportes masiva.
D – Denial of Service	**(24)** Se pueden abusar consultas mal optimizadas para bloquear tablas en Postgres o saturar los canales de lectura.
E – Elevation of Privilege	**N/A**

Instancia de OSRM:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(25)** Se puede enviar peticiones desde la red interna directamente, suplantando al backend.
T – Tampering	**(26)** Alguien con acceso a las instancias puede modificar los archivos de configuración que define la lógica de navegación para cambiar las rutas que se generan.
R – Repudiation	**N/A**
I – Information Disclosure	**N/A**
D – Denial of Service	**(27)** Se pueden pedir consultas que fuercen al motor a consumir muchos recursos (matriz de distancia masiva) de RAM y CPU.
E – Elevation of Privilege	**(28)** Se pueden explotar vulnerabilidades de memoria en librerías que permitan ejecución de código remoto. esto ya que OSRM está escrito en C++.

Instancia VROOM:

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(29)** Se puede enviar peticiones desde la red interna directamente, suplantando al backend.
T – Tampering	**N/A**
R – Repudiation	**N/A**
I – Information Disclosure	**N/A**
D – Denial of Service	**(30)** Se pueden pedir consultas (TSP) que fuercen al motor a consumir muchos recursos de la CPU durante periodos largos de tiempo.
E – Elevation of Privilege	**(31)** Se pueden explotar vulnerabilidades de memoria en librerías que permitan ejecución de código remoto. esto ya que OSRM está escrito en C++.

Redis (Memoria Caché):

STRIDE	Amenaza potencial en este componente
S – Spoofing	**(32)** Se puede simular consultas del backend desde la red interna e inyectar información falsa.
T – Tampering	**N/A**
R – Repudiation	**N/A**
I – Information Disclosure	**(33)** Se puede leer información en tiempo real de los conductores que no esté cifrada y en tránsito.
D – Denial of Service	**(34)** Se puede realizar un ataque que llene la memoria y cause el colapso de Redis
E – Elevation of Privilege	**N/A**

PostgreSQL + PostGIS (Persistencia):

STRIDE	Amenaza potencial en este componente
S – Spoofing	**N/A**
T – Tampering	**(35)** Se puede acceder a la BD o inyectar SQL (si no se valida correctamente). Además de modificar registros enviados desde la app conductor.
R – Repudiation	**(36)** Sí no se hacen copias a un servidor externo se pueden realizar cambios y borrar los logs locales que evidencian el cambio.
I – Information Disclosure	**(37)** Un administrador de la base de datos puede ser convencido de extraer información confidencial si no se definen bien los permisos.
D – Denial of Service	**(38)** Se puede solicitar consultas costosas (principalmente de PostGIS) que dejen a otros usuarios bloqueados por falta de recursos computacionales.
E – Elevation of Privilege	**N/A**

Tabla de resultados

Componente	S	T	R	I	D	E
App Conductor	X	X		X		
Panel Dispatcher	X	X		X	X	
Load Balancer	X	X		X	X	
Backend – Auth	X			X		X
Backend – Ingesta		X			X	
Backend – Core		X	X	X		
Backend – Reportes		X	X	X	X	
Redis	X			X	X	
PostgreSQL		X	X	X	X	
OSRM	X	X			X	X
VROOM	X				X	X

Probabilidad de Compromiso

Por cada amenaza de la parte anterior (se referencias por su índice):

Usamos una escala simple (OWASP Risk Rating Methodology es el estándar de la industria) para calificar la probabilidad de compromiso de la amenaza.

- **ALTA:** El ataque es trivial (hay herramientas públicas), el atacante tiene motivación (ganar dinero) y no tienes controles efectivos.
- **MEDIA:** El ataque requiere conocimientos técnicos medios o herramientas de pago, y tienes algunos controles parciales.
- **BAJA:** Requiere acceso físico difícil, criptografía avanzada o tienes controles robustos (validación geométrica en servidor).

Analizamos cualitativamente con base a:

- **Capacidad de la Amenaza:** ¿Quién es el atacante y qué herramientas tiene?
- **Controls (Defensas):** ¿Qué tan difícil se lo ponemos al atacante, cuales son nuestras defensas?
- La debilidad asociada y su atributo de “explotabilidad” según MITRE.

Índice de caso	**Amenaza**	**Debilidad asociada**	**Controles vs. Capacidad**	**Prob. Final**
1	Spoofing en App conductor	**CWE-602** (Client-side Enforcement) *MITRE: High*	**Capacidad:** Alta **Controles:** ninguno o bajo**,** confianza en cliente.	Alta
2	Tampering en App conductor	**CWE-20** (Improper Input Validation) *MITRE: High*	**Capacidad:** Media **Controles:** ninguno, confianza ciega en el sensor.	Media
3	Info Disclosure en App conductor	**CWE-319** (Cleartext Transmission) & CWE-295 (Improper Certificate Validation) *MITRE: High*	**Capacidad:** Media **Controles:** Altos, se especifica HTTPS/SSL en el Gateway.	Baja

Índice de caso	Amenaza	Debilidad asociada	Controles vs. Capacidad	Prob. Final
4	Spoofing en Panel Dispatcher	**CWE-79** (XSS) / **CWE-1004** (Sensitive Cookie Without 'HttpOnly') *MITRE: High*	**Capacidad:** Media **Controles:** Frameworks protegen contra XSS pero el almacenamiento del navegador es vulnearable	Media
5	Tampering en App conductor	**CWE-602** (Client-Side Enforcement of Server-Side Security) *MITRE: High*	**Capacidad:** Media **Controles:** Depende solo de validación de backend	Alta
6	Info Disclosure en App conductor	**CWE-602** (Client-Side Enforcement of Server-Side Security) MITRE: High	**Capacidad:** Media **Controles:** Se usan protocolos seguros.	Media
7	Info Disclosure en App conductor	**CWE-201** (Insertion of Sensitive Information into Sent Data) MITRE: Medium	**Capacidad:** Alta (El proveedor del mapa posee los logs por diseño). **Controles:** ninguno local, con conexión directa a librería, pero con métodos de seguridad en el servicio	Medio
8	Denial of service en App conductor	**CWE-799** (Improper Control of Interaction Frequency) MITRE: Medium	**Capacidad:** Media **Controles:** Altos. La arquitectura menciona Rate Limiting en el Load balancer.	Baja
9	Spoofing en Load balancer	**CWE-602** (Client-side Enforcement of Server-side Security) *MITRE: High*	**Capacidad:** Media **Controles:** Configuración sólida con con Nginx.	Media
10	Tampering en Load balancer	**CWE-20** (Improper Input Validation) *MITRE: High*	**Capacidad:** Media **Controles:** Altos por configuración de protocolos seguros.	Baja
11	Info disclosure en Load balancer	**CWE-209** (Generation of Error Message Containing Sensitive Info)	**Capacidad:** Media **Controles:** No especifica la arquitectura.	Media

Índice de caso	Amenaza	Debilidad asociada	Controles vs. Capacidad	Prob. Final
		MITRE: Medium		
12	Denial of service en Load balancer	**CWE-400** (Uncontrolled Resource Consumption) *MITRE:High*	**Capacidad:** Alta **Controles:** Nginx tiene límites, pero la red no tiene protección para esto.	Alta
13	Spoofing en módulo Auth	**CWE-287** (Improper Authentication) *MITRE: High*	**Capacidad:** Media **Controles:** Medios.	Media
14	Info disclosure en módulo Auth	**CWE-204** (Observable Response Discrepancy) *MITRE: High*	**Capacidad:** Alta **Controles:** No se especifica.	Alta
15	Elevation of privilege en módulo Auth	**CWE-345** (Insufficient Verification of Data Authenticity) *MITRE: Medium*	**Capacidad:** Media **Controles:** Altos por protecciones del framework, y se especifica seguridad con las JWT.	Baja
16	Tampering en módulo ingesta	**CWE-20** (Improper Input Validation) *MITRE: High*	**Capacidad:** Media **Controles:** Se validan tipos incorrectos por NestJS, pero no se especifican otras validaciones.	Alta
17	Denial of service en módulo ingesta	**CWE-400** (Uncontrolled Resource Consumption) / **CWE-770** (Allocation of Resources Without Limits or Throttling) *MITRE: Medium*	**Capacidad:** Media **Controles:** Medios, se regula por el load balancer, pero no se detallan límites del componente de ingesta.	Media
18	Tampering en Core	**CWE-89** (SQL Injection) **CWE-79** (Cross-Site Scripting)	**Capacidad:** Alta **Controles:** Bajos, el módulo Core realiza validaciones de negocio, por lo que existe control de entrada. SIn embargo	Alta

Índice de caso	Amenaza	Debilidad asociada	Controles vs. Capacidad	Prob. Final
		CWE-78 (OS Command Injection)	no se documentan cuidados técnicos contra inyección.	
19	Repudiation en Core	**CWE-279** (Incorrect Execution-Assigned Permissions) MITRE: Medium **CWE-778** (Insufficient Logging) MITRE: Medium	**Capacidad:** Media **Controles:** No se especifican controles de auditoría o separación de cuentas; se menciona falta de registro y riesgo de cuentas compartidas.	Media
20	Information Disclosure Core	**CWE-209** (Generation of Error Message Containing Sensitive Information) MITRE: Medium	**Capacidad:** Media **Controles:** No se detalla si el sistema captura errores genéricos; se infiere que mensajes directos pueden filtrarse si no se implementa sanitización de errores.	Media
21	Tampering en Reportes	**CWE-472** (External Control of Assumed-Immutable Web Parameter) MITRE: Medium	**Capacidad:** Media **Controles:** No se documentan validaciones para parámetros de reportes ni límites por tipo de reporte.	Media
22	Repudiation en Reportes	**CWE-778** (Insufficient Logging) MITRE: Medium	**Capacidad:** Media **Controles:** Se asocia identidad a las peticiones que pasan por Auth y el Backend (JWT).	Baja
23	information Disclosure en Reportes	**CWE-307** (Improper Restriction of Excessive Authentication Attempts / Rate Limiting Misuse) MITRE: Medium	**Capacidad:** Alta **Controles:** Medio-Bajo, las descargas de reportes están protegidas por Auth. No existe límite de tasa ni controles para evitar extracción masiva.	Media-Alta

Índice de caso	Amenaza	Debilidad asociada	Controles vs. Capacidad	Prob. Final
24	Denial of Service en Reportes	**CWE-1049** (Excessive Data Query Operations in a Large Data Set) MITRE: High/Medium-High **CWE-770** (Allocation of Resources Without Limits or Throttling) MITRE: Medium-High	**Capacidad:** Alta **Controles:** No se recibe tráfico directamente externo, pero no se especifican controles	Alta
25	Spoofing en OSRM	**CWE-346** (Origin Validation Error) MITRE: Medium	**Capacidad:** Media **Controles:** No se especifica.	Media
26	Tampering en OSRM	**CWE-266** (Incorrect Privilege Assignment) MITRE: Medium	**Capacidad:** Alta **Controles:** No se especifica.	Alta
27	Denial of Service en OSRM	**CWE-789** (Memory Allocation with Excessive Size Value) MITRE: High	**Capacidad:** Alta **Controles:** No se tienen limitaciones de tamaño o tiempo por peticion.	Alta
28	Elevation of Privilege en OSRM	**CWE-787** (Out-of-Bounds Write) MITRE: High	**Capacidad:** Media-Alta **Controles:** Medios, los componentes se mantienen aislados.	Media-Baja
29	Spoofing en VROOM	**CWE-346** (Origin Validation Error) MITRE: Medium	**Capacidad:** Media **Controles:** Media, VROOM esta aislado, una suplantación requiere comprometer un nodo interno.	Baja
30	Denial of Service en VROOM	**CWE-407** (Inefficient Algorithmic Complexity) MITRE: High	**Capacidad:** Alta **Controles:** Medios	Media-Alta

Índice de caso	Amenaza	Debilidad asociada	Controles vs. Capacidad	Prob. Final
31	Elevation of Privilege en VROOM	**CWE-787** (Out-of-Bounds Write) MITRE: High	**Capacidad:** Alta **Controles:** No se especifican	Alta
32	Spoofing en Redis	**CWE-346** (Origin Validation Error) MITRE: Medium	**Capacidad:** Media-Alta **Controles:** Aislamiento lógico sin medidas adicionales	Media
33	Information Disclosure en Redis	**CWE-319** (Cleartext Transmission of Sensitive Information) MITRE: High	**Capacidad:** Media **Controles:** No se especifica	Media
34	Denial of Service en Redis	**CWE-789:** (Memory Allocation with Excessive Size Value) MITRE: High	**Capacidad:** Alta **Controles:** No se especifica	Alta
35	Tampering en PostgreSQL	**CWE-89** (SQL Injection) MITRE: High	**Capacidad:** Alta **Controles:** Medios, se valida antes de aceptar operaciones.	Media-Alta
36	Repudiation en PostgreSQL	**CWE-277** (Insecure Inherited Permissions) MITRE: Medium	**Capacidad:** Alta **Controles:** No se especifica	Alta
37	Information Disclosure en PostgreSQL	**CWE-266** (Incorrect Privilege Assignment) MITRE: Medium	**Capacidad:** Media **Controles:** Altos, solo se acepta trafico desde backend y se definen roles con auth.	Baja
38	Denial of Service en PostgreSQL	**CWE-400:** **(**Uncontrolled Resource Consumption) MITRE: High	**Capacidad:** Alta **Controles:** No se especifican restricciones por consulta.	Alta

Análisis de Impacto

El siguiente análisis se plantea como una tabla donde a cada amenaza se le asigna un impacto en forma de costo monetario. Para esto primero se plantea un contexto bajo el cual se realiza el análisis del costo monetario. De esta forma se puede tener una referencia de cómo se ven reflejados estas amenazas en costos para una organización.

Contexto del análisis

Aquí se brinda el contexto operacional de una organización bajo la cual se analizará el impacto monetario de las amenazas:

- **Flota:** 50 camiones en la GAM
- **Costo operativo:** ₡25,000 por hora de inactividad de la plataforma
- **Valor promedio de carga:** ₡2,000,000 por camión
- **Costo Hora Ingeniero (Recuperación/Forense):** ₡30,000/hr.
- **Multas:** ₡50,000 por pedido entregado tarde.

Tabla de análisis de impacto de amenazas

Cada amenaza en esta tabla se referencia con su respectivo índice.

Índice de caso	Amenaza	Impacto (Costo)	Detalle del impacto
1	Spoofing en App conductor	₡ 150,000	Pago de horas extra y combustible por rutas falsificadas no recorridas realmente.
2	Tampering en App conductor	₡ 450,000	Mantenimiento correctivo y daños por frenazos no detectados.
3	Info Disclosure en App conductor	₡ 2,000,000	Robo total de la carga al interceptar ruta y contenido de alto valor.
4	Spoofing en Panel Dispatcher	₡ 500,000	Auditoría forense y revocación de credenciales tras acceso no autorizado de terceros.
5	Tampering en Panel Dispatcher	₡ 200,000	Pérdida de combustible y tiempo por re-enrutamiento malicioso de unidades activas.

Índice de caso	Amenaza	Impacto (Costo)	Detalle del impacto
6	Info Disclosure en Panel Dispatcher	₡ 1,500,000	Fuga de inteligencia comercial (rutas/clientes) hacia la competencia directa.
7	Info Disclosure en Panel Dispatcher	₡ 300,000	Fuga leve de inteligencia comercial
8	Denial of service en Panel Dispatcher	₡ 100,000	Aumento de costo operativo por gestión manual (sin sistema)
9	Spoofing en Load balancer	₡ 60,000	Costos por re-procesamiento de órdenes y reintentos de envío
10	Tampering en Load balancer	₡ 300,000	Desvío de rutas, retrasos y necesidad de reasignación de entregas.
11	Info disclosure en Load balancer	₡ 150,000	Exposición de información interna en que obliga investigación y soporte
12	Denial of service en Load balancer	₡ 600,000	Caída parcial del servidor. Afectación a la operación diaria y tiempo muerto de trabajo.
13	Spoofing en módulo Auth	₡ 800,000	Costo de reasignaciones, pérdida de trazabilidad y entregas fallidas
14	Info disclosure en módulo Auth	₡ 1,500,000	Costo de compensaciones y esfuerzos de contención de información
15	Elevation of privilege en módulo Auth	₡ 5,000,000	Reasignaciones masivas, cancelación de entregas, generación de reportes falsos, pérdida operativa y costos de restauración de la información
16	Tampering en módulo ingesta	₡ 300,000	Consumo adicional de combustible y correcciones manuales.
17	Denial of service en módulo ingesta	₡ 125,000	Pérdida de tiempo operativo por falta de trazabilidad.
18	Tampering en Core	₡ 1,000,000	Restauración de base de datos.

Índice de caso	Amenaza	Impacto (Costo)	Detalle del impacto
19	Repudiation en Core	₡ 400,000	Fraude/robo interno.
20	Information Disclosure Core	₡ 300,000	Filtración de logs y parcheo (tiempo técnico)
21	Tampering en Reportes	₡ 600,000	Pagos a conductores con reportes de rendimiento alterados.
22	Repudiation en Reportes	₡ 50,000	Tiempo operativo.
23	information Disclosure en Reportes	₡ 5,000,000	Filtración de datos, multas regulatorias y posibles demandas.
24	Denial of Service en Reportes	₡ 100,000	Bloqueo temporal de reportes.
25	Spoofing en OSRM	₡ 30,000	Consumo de recursos computacionales.
26	Tampering en OSRM	₡ 300,000	Incremento en costos de combustible.
27	Denial of Service en OSRM	₡ 150,000	Retrasos en las entregas.
28	Elevation of Privilege en OSRM	₡ 1,500,000	Posible escape al Host, daño a infraestructura.
29	Spoofing en VROOM	₡ 30,000	Consumo de recursos computacionales..
30	Denial of Service en VROOM	₡ 150,000	Retrasos en las entregas.
31	Elevation of Privilege en VROOM	₡ 1,500,000	Posible escape al Host, daño a infraestructura.
32	Spoofing en Redis	₡ 50,000	Confusión operativa momentánea.
33	Information Disclosure en Redis	₡ 2,000,000	Intercepción de ubicaciones facilitando asaltos a camiones.

Índice de caso	Amenaza	Impacto (Costo)	Detalle del impacto
34	Denial of Service en Redis	₡ 75,000	Molestia por latencia.
35	Tampering en PostgreSQL	₡ 1,500,000	Limpieza manual de datos, posible detención de operación
36	Repudiation en PostgreSQL	₡ 600,000	Robos internos sin poder sancionar
37	Information Disclosure en PostgreSQL	₡ 10,000,000	Robo de base de datos completa
38	Denial of Service en PostgreSQL	₡ 500,000	Paro en la empresa por no poder leer ni escribir datos

Priorización de riesgos

Cálculo del riesgo

El cálculo del riesgo se realizará mediante la evaluación de los daños potenciales agravándose en base a la probabilidad de que estos casos ocurran. Se utiliza un sistema de puntuación en el que se asigna mayor cantidad de puntos a los riesgos más elevados, los cuales permiten identificar la amenaza más severa. Se divide entre 1000, para una mejor visualización de los resultados

Fórmula:

$$Riesgo = \frac{Costo\ Base \times Probabilidad\ Final}{1000}$$

Dado que en este análisis se utilizó la probabilidad como una propiedad cualitativa se asigna un sistema de puntos para las probabilidades de la siguiente forma, asumiendo los puntos medios:

- Baja = 1
- Media = 3

- Alta = 5

A partir de esto se tomarán como riesgos prioritarios aquellos que tengan un valor superior a 2000. Estos se verán reflejados en la tabla con otro color.

Tabla de riesgos

Índice	Amenaza	Impacto (Costo)	Probabilidad	Riesgo
37	Information Disclosure en PostgreSQL	₡ 10,000,000	Baja	10000
14	Info disclosure en módulo Auth	₡ 1,500,000	Alta	7500
31	Elevation of Privilege en VROOM	₡ 1,500,000	Alta	7500
33	Information Disclosure en Redis	₡ 2,000,000	Media	6000
35	Tampering en PostgreSQL	₡ 1,500,000	Media-Alta	6000
15	Elevation of privilege en módulo Auth	₡ 5,000,000	Baja	5000
18	Tampering en Core	₡ 1,000,000	Alta	5000
6	Info Disclosure en Panel Dispatcher	₡ 1,500,000	Media	4500
23	information Disclosure en Reportes	₡ 5,000,000	Media-Alta	4000
28	Elevation of Privilege en OSRM	₡ 1,500,000	Media- Baja	3000
36	Repudiation en PostgreSQL	₡ 600,000	Alta	3000
38	Denial of Service en PostgreSQL	₡ 500,000	Alta	2500
13	Spoofing en módulo Auth	₡ 800,000	Media	2400
21	Tampering en Reportes	₡ 600,000	Media	1800
4	Spoofing en Panel Dispatcher	₡ 500,000	Media	1500
16	Tampering en módulo ingesta	₡ 300,000	Alta	1500

Índice	Amenaza	Impacto (Costo)	Probabilidad	Riesgo
26	Tampering en OSRM	₡ 300,000	Alta	1500
2	Tampering en App conductor	₡ 450,000	Media	1350
12	Denial of service en Load balancer	₡ 600,000	Alta	1200
5	Tampering en Panel Dispatcher	₡ 200,000	Alta	1000
7	Info Disclosure en Panel Dispatcher	₡ 300,000	Medio	900
20	Information Disclosure Core	₡ 300,000	Media	900
1	Spoofing en App conductor	₡ 150,000	Alta	750
27	Denial of Service en OSRM	₡ 150,000	Alta	750
30	Denial of Service en VROOM	₡ 150,000	Media-Alta	600
24	Denial of Service en Reportes	₡ 100,000	Alta	500
11	Info disclosure en Load balancer	₡ 150,000	Media	450
17	Denial of service en módulo ingesta	₡ 125,000	Media	375
34	Denial of Service en Redis	₡ 75,000	Alta	375
10	Tampering en Load balancer	₡ 300,000	Baja	300
3	Info Disclosure en App conductor	₡ 2,000,000	Baja	200
9	Spoofing en Load balancer	₡ 60,000	Media	180
32	Spoofing en Redis	₡ 50,000	Media	150
19	Repudiation en Core	₡ 400,000	Media	120
8	Denial of service en Panel Dispatcher	₡ 100,000	Baja	100
25	Spoofing en OSRM	₡ 30,000	Media	90
22	Repudiation en Reportes	₡ 50,000	Baja	50

Índice	Amenaza	Impacto (Costo)	Probabilidad	Riesgo
29	Spoofing en VROOM	₡ 30,000	Baja	30

Plan de Mitigación de Riesgos

Las mitigaciones de este plan se centrarán en tratar con las amenazas que tengan un valor de riesgo superior a **2000** con el fin de acotar el alcance del desarrollo y cuidar los recursos.

Mitigación par casos seleccionados en base al riesgo

Information Disclosure en PostgreSQL (37)

Caso: Filtración de información sensible por consultas mal diseñadas, errores directos expuestos, configuración interna o metadatos revelados

Objetivo: Prevenir la exposición accidental o directa de información almacenada en PostgreSQL.

Medidas de mitigación:

- Configurar el backend para capturar internamente cualquier error generado por PostgreSQL y reemplazarlo por un mensaje de error genérico antes de devolverlo al usuario para evitar exponer detalles del funcionamiento interno.
- Ajustar los roles y permisos dentro de PostgreSQL para que solo se acceda a las tablas estrictamente necesarias y eliminar derechos de lectura sobre datos de estructura interna de la base de datos.
- Realizar un cifrado de conexión entre el backend y PostgreSQL, para asegurar que ninguna información se pueda filtrar por medio de captura de tráfico interno.
- Optimizar las consultas del backend para devolver únicamente los campos necesarios

Info disclosure en módulo Auth (14)

Caso: La API de autenticación devuelve mensajes y estados que permiten identificar usuarios y exponen información interna del login.

Objetivo: Evitar la filtración de datos sensibles relacionados al proceso de autenticación

Medidas de mitigación:

- Modificar el módulo para siempre devolver mensajes de error genéricos, sin revelar la existencia de usuarios
- Integrar un sistema de rate limiting a las rutas de login del módulo Auth

Elevation of Privilege en VROOM (31)

Caso: uso indebido de endpoints para provocar ejecución de funciones privilegiadas o comportamiento no autorizado.

Objetivo: Impedir que un atacante pueda interactuar con VROOM de forma que obtenga resultados o acciones no autorizadas.

Medidas de mitigación:

- Configurar reglas de red internas para limitar la conexión con VROOM
- Establecer que no se puedan añadir nuevos privilegios dentro del contenedor de VROOM
- implementar autenticación de servicio a servicio mediante certificados internos.

Information Disclosure en Redis (33)

Caso: Acceso no autorizado a datos en memoria debido a falta de autenticación, cifrado o restricciones de red.

Objetivo: Evitar que datos sensibles almacenados en forma de chaché puedan ser accedido o interceptados

Medidas de mitigación:

- habilitar autenticación interna en Redis mediante contraseñas o ACLs para evitar que cualquier proceso dentro de la red interna pueda conectarse sin credenciales válidas.
- Configurar Redis para que escuche únicamente dentro de la red privada y sea accesible únicamente por el backend.
- Activar cifrado en conexiones internas hacia Redis

Tampering en PostgreSQL (35)

Caso: Modificación no autorizada de datos mediante inyección SQL, manipulación de inputs o abusos del rol asignado al backend.

Objetivo: Garantizar la integridad de la información registrada en la base de datos

Medidas de mitigación:

- Convertir todas las consultas dinámicas del backend a consultas parametrizadas.
- Ajustar los roles de PostgreSQL para que el servicio de backend funcione con privilegios minimos

Elevation of privilege en módulo Auth (15)

Caso: Manipulación de tokens JWT o validaciones defectuosas que permiten obtener privilegios o roles superiores a los autorizados.

Objetivo: Impedir que un atacante pueda escalar niveles de acceso mediante el sistema de autenticación.

Medidas de mitigación:

- Validar exhaustivamente el contenido de cada token recibido.
- Mantener un corto tiempo de vida para los tokens
- Redefinir el sistema de autorización para que no dependa únicamente del contenido de los tokens aplicando comprobaciones adicionales.

Tampering en Core (18)

Caso: Manipulación de solicitudes o parámetros recibidos por el COre que permite alterar valores y consultas internas

Objetivo: Evitar que el core procese datos manipulados que alteren el comportamiento del sistema.

Medidas de mitigación:

- Implementar un sistema de validación de entradas para que el Core rechace cualquier solicitud de contenido que no cumpla los formatos esperados.
- Utilizar mecanismos como firmas de mensajes o verificaciones de contenido para garantizar que los datos transmitidos no hayan sido alterados y controlar la integridad del intercambio de información con otros componentes.

Info Disclosure en Panel Dispatcher (6)

Caso: Información interceptada por medio de la red compartida.

Objetivo: Ocultar información a terceros y dificultar la extracción de datos.

Medidas de mitigación:

- Utilizar un Proxy para realizar las llamadas a librerías de terceros por medio de un servicio intermedio que anonimize el tráfico.
- Usar solo protocolos seguros como HTTPS.

information Disclosure en Reportes (23)

Caso: Acceso a reportes por usuarios no autorizados mediante URLs y descargas residuales.

Objetivo: Asegurar acceso exclusivo y eliminar persistencia de archivos en disco.

Medidas de mitigación:

- Enviar todo archivo generado directamente y no guardar ni escribir archivos físicos en el servidor.
- Implementar métodos que verifiquen la propiedad de la información, revisando no solo el rol pero que los IDs del usuario solicitante y el usuario dueño coinciden.

Elevation of Privilege en OSRM (28)

Caso: Explotación del binario C++ de OSRM para escapar del contenedor.

Objetivo: Contener el proceso y evitar acceso al sistema principal.

Medidas de mitigación:

- Configurar el contenedor para que la aplicación nunca corra como *root*.
- Configurar el contenedor para que no tenga permisos del kernel para montar volúmenes o manipular la red.
- Validar las entradas en la capa de aplicación para evitar el ingreso de datos no esperados, de tipos diferentes a las esperadas y con límites a la cantidad de puntos para la ruta.

Repudiation en PostgreSQL (36)

Caso: Eliminación de registros críticos y logs locales que identifiquen culpables.

Objetivo: Garantizar la trazabilidad de toda manipulación.

Medidas de mitigación:

- Configurar Postgres para enviar logs a un servidor externo en tiempo real de forma persistente.

- Habilitar registros detallados (con extensiones) para operaciones de modificación de datos y eliminación de tablas.
- Prohibir los `DELETE` físicos y mantener un historial en la base de cualquier deshabilitación de información.

Denial of Service en PostgreSQL (38)

Caso: Consultas geométricas complejas saturan la CPI/IO.

Objetivo: Aislar la carga de reportes y limitar recursos para proteger la operación.

Medidas de mitigación:

- Configurar un ***time out*** para las consultas de forma que la base de datos mate los procesos para liberar recursos.
- Asegurar que los reportes complejos solo se ejecuten automáticamente en tiempos no laborables y de poco tráfico.

Spoofing en módulo Auth (13)

Caso: Robo de Token JWT almacenado inseguramente permite suplantar identidad de usuarios válidos

Objetivo: Prevenir el robo de credenciales y minimizar el tiempo de acceso de sesiones comprometidas.

Medidas de mitigación:

- Dejar de enviar el JWT en el cuerpo de la respuesta y hacerlo en forma de cookie con flags de `HttpOnly` que solo viaje por HTTPS. De esa forma JS no puede leerla y viaja por protocolo seguro que evita el robo en tránsito.
- Cada token dura 15 minutos antes de ser rotado y los *refresh tokens* se invalidan si es usado 2 veces (por usuario y atacante). Obligando al loguearse de nuevo.

Hallazgos

Durante el análisis de la arquitectura se identificaron varios riesgos importantes relacionados con cómo los componentes manejan datos, validan información y se comunican entre sí. El módulo donde se encontraron más problemas fue Auth, especialmente por dos situaciones: por un lado, algunos mensajes y errores podrían revelar

información sensible; por otro, ciertas validaciones son débiles y permitirían que un atacante altere o use tokens para obtener permisos que no le corresponden.

En el módulo Core también se detectaron riesgos de manipulación de datos. Si los parámetros de entrada no se validan bien, un atacante podría modificar información o influir en las consultas internas que el Core envía a otros servicios como PostgreSQL o VROOM.

En cuanto a los servicios auxiliares, tanto PostgreSQL como Redis muestran riesgos de exposición de información. PostgreSQL podría filtrar detalles internos a través de errores o consultas mal diseñadas, mientras que Redis podría ser accedido sin autorización si no se agregan controles adicionales dentro de la red interna.

Finalmente, en VROOM se identificó un riesgo de escalación de privilegios debido a la falta de autenticación entre servicios y a la manera en que el contenedor ejecuta ciertas funciones. Esto podría permitir que una solicitud alterada afecte las rutas o cálculos que realiza el motor.

En general, los hallazgos indican que la arquitectura necesita reforzar las validaciones, mejorar el manejo de errores y asegurarse de que la comunicación entre servicios esté protegida. Si estos puntos no se atienden, podrían derivar en accesos indebidos, exposición de información o fallas en la operación del sistema.